

Semivra Libellus

Enterprise POS + ERP - Independent Production-Readiness Audit

OVERALL PRODUCTION READINESS



Verdict: RELEASE CANDIDATE - deployable to a single-tenant SMB after 3 must-fix blockers are cleared.

304 / 304 SERVER TESTS PASSING	77.8% SERVER LINE COVERAGE	139 API ROUTES - ALL RBAC-GATED
9 DEPENDENCY VULNERABILITIES	3 MUST-FIX DEPLOY BLOCKERS	~25.5k LINES OF APPLICATION CODE

Application	Semivra Libellus - QR menu / POS / double-entry ERP (dual mode: F&B + Logistics)
Stack	React 18 + Vite 8 + Tailwind Node/Express 4 + MongoDB (Mongoose 9) + Socket.io
Repository	Branch pwa-rbac-followups; build clean; 304 tests green; working tree has uncommitted changes
Audit date	2026-06-30 - evidence-based; tests, coverage, build & dependency audits executed
Auditor	Independent review - Architect / QA / DevOps / Security / Performance / UX
Method	Full static review + executed: vitest (304), v8 coverage, vite build, npm audit, Playwright e2e (9/9 in real Chromium)

Table of Contents

1. Executive Summary	3
2. Project Overview	3
3. Architecture Review	4
4. Version Analysis	5
5. Change Log	5
6. Feature Audit	6
7. Code Quality Review	6
8. Error Detection	7
9. Dependency Audit	8
10. Security Audit	9
11. Performance Audit	10
12. UI / UX Review	11
13. Database Review	11
14. API Review	11
15. Testing Review	12
16. Deployment Review	13
17. Production Readiness Ratings	14
18. Deployment Decision	14
19. Bug Prediction & Risk Analysis	15
20. Completeness Analysis	15
21. Final Scorecard	16
22. Final Verdict & 100% Completion Checklist	16
Appendix A - What To Read Later	17

1. Executive Summary

Semivra Libellus is a full-stack, offline-capable Point-of-Sale and back-office ERP for Philippine small businesses, with two modes chosen by a `BUSINESS_TYPE` env flag: **F&B** (QR self-ordering cafe/restaurant) and **Logistics** (client-login ordering with billing statements). It unifies POS, recipe/BOM inventory with FEFO batch expiry, genuine double-entry accounting with BIR-aware Philippine tax, shift/cash-drawer control, and a PWA offline queue into one tablet-first app.

This is not a prototype. The backend is genuinely well-engineered: dual-token auth (in-memory access token + `httpOnly`, server-revocable refresh cookie), Zod boundary validation with mass-assignment defense, Helmet, granular role-based authorization on all 139 routes, MongoDB transactions guarding accounting integrity, balanced-journal assertions, atomic counters/inventory, a production-safe centralized error handler, graceful shutdown, and structured logging. It ships Docker, docker-compose, a CI workflow, backup/restore scripts, and 304 passing server tests.

Can it be deployed today? YES - conditionally.

The app builds cleanly, all 304 server tests pass, and the deployment scaffolding (Docker, Vercel/Railway config, /health, graceful shutdown) is in place. Three issues must be fixed BEFORE any real deployment: (1) the live JWT signing secret is weak and reused, (2) high-severity dependency vulnerabilities are unpatched, (3) an unauthenticated endpoint exposes customer PII. None require architectural change; all are fixable in well under a day.

Top strengths (verified)

- **Real double-entry accounting** - every sale/void/spoilage/expense posts a balanced journal entry; an assertion throws if `DR != CR`; order completion runs inside a MongoDB transaction.
- **Disciplined authorization** - all 139 routes enumerated; every sensitive route carries `verifyToken` + `requireStaff/requireSuperAdmin`; Socket.io decides rooms server-side (no self-elevation).
- **Mature auth** - access token in memory only (XSS-resistant); refresh token `httpOnly`, hashed at rest, rotated on use, revoked on logout/password/role change.
- **Backend test depth** - 304 tests / 20 files incl. fault injection, money math, edge cases; 77.8% line coverage.

Top weaknesses (verified)

- **Weak, shared JWT secret** in the live `.env` (`infu-pass-2026`); boot validates presence, not strength.
- **Unauthenticated** `GET /api/orders/:id` returns customer name, phone, delivery address (IDOR / PII).
- **Public products endpoint leaks COGS/recipe (BOM)** data to anonymous customers.
- **9 dependency vulnerabilities** (3 high server / 2 high client - mostly WS DoS); CI audit gate is non-blocking.
- **Maintainability debt** - `server.js` 6,691 lines; `AdminDashboard.jsx` 6,098 lines / 234 `useState`; no ESLint/Prettier; stale in-repo audit docs contradict current code.

2. Project Overview

What it is and the problem it solves

Semivra Libellus replaces the "generic POS + manual spreadsheet bookkeeping" workflow of a PH micro/small business with a single system that "closes its own books": each transaction is auto-journalized into a chart of accounts, producing export-ready P&L, Balance Sheet, A/R and percentage-tax reports without re-keying.

Target users

- **F&B mode** - cafe / restaurant operators wanting QR self-ordering, kitchen/bar routing, recipe COGS, and cash-drawer control.
- **Logistics mode** - small trading/logistics operators billing named client accounts with statements, deposits, and partial fulfillment.

- **Roles** - superadmin (owner), admin/manager, cashier/staff, and authenticated clients (logistics portal).

Technology, frameworks, libraries, services (verified from package.json + installed)

Layer	Technology (installed version)
Frontend	React 18.3.1, Vite 8.0.16, Tailwind 3.4, react-router-dom 6.30, lucide-react 1.17, socket.io-client 4
Exports	jspdf 4.2.1 + jspdf-autotable 5.0.8, SheetJS xlsx 0.20.3 (CDN tarball), html2canvas, react-qr-code 2.0.21
Backend	Node 22 / Express 4.22, Mongoose 9.6, Socket.io 4.8.3, jsonwebtoken 9.0.3, bcrypt 6.0
Security/infra	helmet 8.2, express-rate-limit 8, cookie-parser, cors, zod 4.4, compression
Observability	pino 10 + pino-http (structured logs); optional @sentry/node 10.56 (inert unless DSN set)
Database	MongoDB Atlas (cloud); transactions require a replica set - Atlas provides this
Testing	vitest 4 + @vitest/coverage-v8, mongodb-memory-server, supertest; Playwright (client e2e)
Deploy	Vercel (SPA) + Railway (API); Docker + docker-compose + nginx for self-host

3. Architecture Review

- **SPA + JSON API + WebSocket.** React SPA (lazy-loaded routes) talks to a stateless Express JSON API; Socket.io pushes real-time order/menu updates. The API serves no HTML (CSP intentionally off server-side; belongs on the SPA host).
- **Pure-logic core, unit-tested.** server/lib/ holds framework-free modules - ledger, tax, units, expiry, chartOfAccounts, authz, reportRange - each with a co-located *.test.js. Money and authorization decisions are testable without booting Express/Mongo. This is the single best architectural decision in the codebase.
- **Layered request pipeline.** helmet -> compression -> CORS -> pino-http -> body parsers -> cookie-parser -> per-route (rate limit -> verifyToken -> requireStaff/SuperAdmin -> Zod validate -> handler -> emit) -> 404 -> centralized error handler.
- **Single-tenant per deployment.** Data is scoped by a businessType field, not true multi-tenancy. A multi-branch rollout would require tenant scoping + Socket.io room partitioning.
- **Monolithic delivery files.** Nearly all backend logic is in one 6,691-line server.js; the admin UI is one 6,098-line component. Functional, but a maintainability/merge-risk liability (see Code Quality).

Source size map (verified - wc -l)

File	Lines	Role
server/server.js	6,691	All routes, schemas, ledger orchestration, sockets
client/.../AdminDashboard.jsx	6,098	Admin SPA shell (234 useState, 11 useEffect)
client/.../tabs/LedgerTab.jsx	1,928	Accounting UI
client/.../tabs/OrdersTab.jsx	1,277	POS / orders UI
client/.../pages/CustomerMenu.jsx	1,043	QR customer ordering
server/lib/*.js (7 modules)	~620	Pure, fully unit-tested core logic

4. Version Analysis

Version identifiers disagree across the repo - a finding in itself. There are no git tags.

Source	Declared version
server/package.json, client/package.json	1.0.0
README.md "Version History"	v0.2 (ERP & Management Upgrade)
Recent git commit subjects	"new version 0.5", "v.5", "0.9/1.1 fix patch"
Git tags	none

Evolution timeline (from git history + in-repo reports)

- **v0.1** - Core QR menu + real-time kitchen display (WebSockets), menu builder, basic order lifecycle, manual discount/VAT.
- **v0.2** - ERP upgrade: double-entry accounting, weighted-average inventory, BOM engine, analytics, CSV export, PWA wake-lock.
- **Security hardening (PR #1, branch security/hardening-p1-p3)** - Helmet, Zod, bcrypt 10->12, regex-injection fix, centralized error handler, and the **breaking dual-token auth rewrite**.
- **PWA + RBAC follow-ups (current branch)** - offline POS/clock queue, RBAC void/refund split, responsive sidebar, the logistics (Log) mode with client portal + billing statements.

Architecture / dependency / config changes across versions

- Auth: single localStorage JWT -> dual-token (in-memory access + httpOnly refresh + server revocation).
- Deps added over time: helmet, zod, cookie-parser, @sentry/node, express-rate-limit, pino; client added Playwright; xlsx swapped from abandoned npm package to patched SheetJS 0.20.3 CDN build; Vite 5 -> 8.
- Config: introduced BUSINESS_TYPE, ALLOWED_ORIGINS, SENTRY_DSN, billing-statement VITE_* vars; Docker/compose/CI added.

5. Change Log

Current branch vs origin/main (git diff - committed)

pwa-rbac-followups vs origin/main: **3 files, +68 / -23** (PricingTab.jsx, AdminDashboard.jsx, server.js).

Uncommitted working-tree changes (git status / diff - NOT yet committed)

23 files, +1,855 / -597. Largest: server.js (+585/-), AdminDashboard.jsx (~183 lines), all 8 tab components, index.css, plus a server dependency bump (1,387-line package-lock churn).

Category	Change
Backend	server.js heavily modified (uncommitted); new lib modules authz.js, tax.js, reportRange.js + tests
Tests	New server/test/ integration suite (15 files) + vitest.config.js
Frontend	All tab components + AdminDashboard, SuperAdminPanel, CustomerMenu, Client* pages modified
Dependencies	server/package.json + package-lock churn; root package.json scripts

Docs	AUDIT_OVERVIEW.md, README.md, SESSION_REPORT.md edited
Config	No tracked .env (gitignored); CI, Docker, compose present

Risk: substantial uncommitted work.

A large body of changes (incl. core server logic and dependencies) lives only in the working tree - unbacked-up, unreviewed, and NOT what CI validated on the branch. Commit to a feature branch and let CI run before relying on it.

6. Feature Audit

Legend: GOOD = implemented & evidenced. PARTIAL = present but caveated. MISSING = absent/stub.

Feature	Status	Evidence / notes
POS register, checkout, split payments	GOOD	Order POST w/ idempotency key, payments[]; OrdersTab 1,277 LOC
QR self-ordering (F&B)	GOOD	Crypto-random session, time-boxed, single-use "burn" on order
Client-login ordering + billing (Logistics)	GOOD	verifyClientToken gate; client-portal integration test
Double-entry GL	GOOD	Balanced-journal assertion; txn-wrapped; lib/ledger unit-tested
PH percentage tax (3%) / VAT modes	GOOD	lib/tax.js pure + tested; gross-receipts model
Inventory: recipe deduction + FEFO expiry	GOOD	lib/expiry.js FEFO; atomic \$inc deduction
Excel/CSV import & stock-take	GOOD	SheetJS 0.20.3; diff-preview; journal-backed
Reports: P&L, Balance Sheet, A/R, tax	GOOD	Mongo aggregation; date-range bounded (max 92d)
Shifts / cash drawer / EOD reconcile	GOOD	shift cash filter; variance report
Real-time multi-device sync	GOOD	Socket.io JWT handshake + server-decided rooms
PWA offline order/clock queue	GOOD	lib/pwa.js + service worker (network-first/SWR)
RBAC (superadmin/admin/staff/client)	GOOD	All 139 routes; void=superadmin, refund=super/admin
Revolving/petty cash fund	GOOD	disburse/replenish/close routes + ledger
Push "order ready" notification	PARTIAL	In-page showNotification only; no VAPID server push
Analytics dashboard	PARTIAL	Works but computed client-side; will not scale to large data
Multi-branch / franchise rollup	MISSING	Single-tenant by businessType field only
Loyalty / CRM	MISSING	Acknowledged roadmap item
Hardware printer / cash-drawer (ESC/POS)	MISSING	Browser print only; no certification

7. Code Quality Review

Strengths

- **Excellent separation of pure logic** into `server/lib/*` with co-located tests - the highest-risk logic (money, tax, authz) is isolated and verifiable.
- **Consistent defensive route pattern**: `validate -> authorize -> transaction -> emit -> typed {success, ...}` response, with prod-safe errors.
- **Thoughtful "why" comments** (A/R vs cash-on-hand policy, aud-beats-role precedence, SameSite/CSRF trade-off) - above average for an SMB project.

Issues

MEDIUM God files / monolith

Evidence: `server.js` 6,691 lines; `AdminDashboard.jsx` 6,098 lines / 234 `useState`; `LedgerTab` 1,928; `OrdersTab` 1,277.

Impact: High cognitive load, painful merges, hard-to-test UI, slow onboarding, easy regressions.

Fix: Continue the started components/tabs extraction; split `server.js` into route modules behind the lib core.

MEDIUM No linter or formatter

Evidence: No `.eslintrc` / `eslint.config` / `.prettierrc` anywhere; CI runs only `node --check`.

Impact: No automated catch of unused vars, accidental globals, missing hook deps, style drift.

Fix: Add ESLint (+react-hooks plugin) + Prettier and a CI lint step.

LOW Dead / temp artifacts committed

Evidence: `scripts_temp/` holds 10 one-off dev scripts; `showcase.html` duplicated at repo root and `client/public/`; 8 generated PDFs committed.

Impact: Repo clutter; confusion over source vs scratch; larger clone.

Fix: Move throwaway scripts out (or to `/tools` with a README); de-dupe `showcase.html`; gitignore generated PDFs.

LOW Mongoose deprecation warnings

Evidence: Test output repeats "the new option for `findOneAndUpdate()` is deprecated".

Impact: Forward-compat risk on a future Mongoose major; log noise.

Fix: Replace `{ new: true }` with `{ returnDocument: "after" }`.

LOW Unconventional in-file structure

Evidence: Schemas, routes, helpers, and seeding are interleaved in `server.js`; some helpers defined after first use (hoisting-dependent).

Impact: Harder to navigate/review.

Fix: Group by concern or split into modules.

8. Error Detection

No compilation/build errors were found (`server node --check` passes in CI; `vite build clean`). The runtime/logic risks below were identified by inspection.

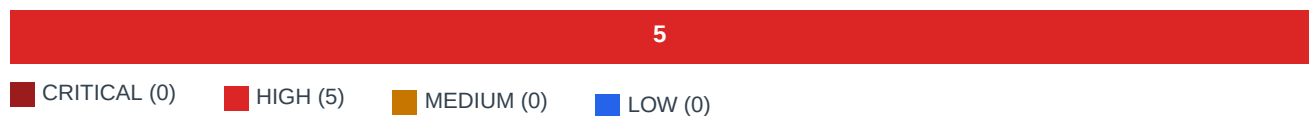
Area	Severity	Finding
Auth	HIGH	GET <code>/api/orders/:id</code> has no auth guard - reachable without a token (see Security).
Data exposure	MEDIUM	Public <code>/api/products</code> returns <code>recipe/cost</code> fields to anonymous callers.
DB dependency	MEDIUM	Multi-doc transactions throw on a non-replica-set Mongo (standalone/local).

Async	LOW	Startup seeding wrapped in try/catch and logs errors but proceeds; a partial seed failure is non-fatal (intended, but can mask issues).
Validation	LOW	Some write routes still use <code>Model.create(req.body)</code> ; Mongoose strict mode drops unknown keys so risk is low, but not all routes have explicit Zod schemas.
Deprecation	LOW	Mongoose <code>{ new: true }</code> usage will break on a future major.
Unhandled input	LOW	Most boundaries covered by Zod + <code>escapeRegex</code> ; analytics math assumes well-formed numeric aggregates.

Crash handling is robust: `uncaughtException` / `unhandledRejection` are caught, logged (and sent to Sentry if configured), then the process drains and exits for a supervisor restart - the correct pattern.

9. Dependency Audit

`npm audit` was executed in both packages during this review.



(5 HIGH = 3 server + 2 client; plus 4 moderate not shown in the bar. 9 total.)

Package (transitive)	Where	Severity	Advisory
ws 8.0-8.20.1	server + client	HIGH	Memory-exhaustion DoS (via <code>socket.io</code> / <code>engine.io</code>)
@opentelemetry/core	server	MODERATE	Unbounded memory in W3C baggage (via <code>@sentry/node</code>)
dompurify <=3.4.10	client	MODERATE	Trusted-Types / <code>ALLOWED_ATTR</code> pollution (via <code>jspdf</code>)

- **All are fixable** via `npm audit fix` in `server/` and `client/` (no breaking majors required for the high-severity ones).
- **CI gate is non-blocking** - the workflow ends the audit step with `|| echo "...review."`, so advisories never fail the build.
- **Direct dependencies are current and legitimate** - no abandoned or typosquatted packages observed; the prior abandoned `xlsx` was already swapped for patched SheetJS 0.20.3.

Direct dependency inventory

Server (prod)	Client (prod)
express 4.22, mongoose 9.6, socket.io 4.8	react 18.3, react-dom, react-router-dom 6.30
jsonwebtoken 9, bcrypt 6, zod 4.4	socket.io-client 4, lucide-react 1.17
helmet 8.2, cors, express-rate-limit 8	jspdf 4.2 + autotable 5, xlsx 0.20.3, html2canvas
compression, cookie-parser, dotenv	react-qr-code 2
pino/pino-http, @sentry/node 10.56	dev: vite 8, tailwind 3.4, @playwright/test 1.6

10. Security Audit

Security DESIGN is strong; the gaps are CONFIGURATION (weak secret), a couple of EXPOSURE points, and DEPENDENCIES. Finding severity distribution:



CRITICAL Weak and reused JWT signing secret

Evidence: Live .env sets JWT_SECRET=infu-pass-2026 - short, guessable, dictionary-style - identical across the F&B and logistics deployments. Boot only checks presence (server.js:44), not strength.

Impact: Anyone who guesses/obtains it can forge a superadmin token and bypass all auth; reuse means one leak compromises both businesses.

Fix: openssl rand -hex 32, UNIQUE per deployment, store in host secret manager, add a boot check rejecting secrets < 32 chars, rotate now.

HIGH Unauthenticated order lookup exposes PII (IDOR)

Evidence: GET /api/orders/:id (server.js:2085) has NO auth and returns customerName, customerPhone, deliveryAddress, totals, payment method.

Impact: MongoDB ObjectIds embed a timestamp + counter and are partly predictable; orders can be enumerated to harvest PII (OWASP A01).

Fix: Require a matching QR-session/client token, or return only non-PII status fields on the public path.

HIGH Unpatched high-severity dependencies

Evidence: ws memory-exhaustion DoS reachable via Socket.io on both server and client (npm audit, executed).

Impact: Remote DoS vector on any deployment.

Fix: npm audit fix in both packages; make CI audit blocking for high/critical.

MEDIUM Internal cost / recipe (COGS) data exposed publicly

Evidence: Public GET /api/products strips clientDiscounts/images for anonymous callers but still returns baseRecipe, costOverride, and sizes[].recipe / addOns[].recipe costs.

Impact: Competitors/customers read ingredient lists, per-item costs, and margins via the menu.

Fix: Project only customer-facing fields for unauthenticated/QR callers.

MEDIUM Guessable / default admin credentials

Evidence: Seed fallback ChangeMe@2026! (server.js:405, docker-compose default); live ADMIN_PASS follows a guessable pattern (...masterkey2026!).

Fix: Force a strong unique admin password on first boot; never ship a default.

MEDIUM No secret-strength / config validation at boot

Evidence: Only presence of MONGO_URI & JWT_SECRET checked; weak secret, default admin pass, or empty prod ALLOWED_ORIGINS accepted silently.

Fix: Add a startup config validator.

MEDIUM Non-blocking security gate in CI

Evidence: ci.yml audit step ends with || echo so advisories never fail the build.

Fix: Fail CI on high/critical with a time-boxed allowlist for accepted advisories.

LOW Client-side route guards absent

Evidence: /admin & /admin/admin-panel render directly in App.jsx; protection relies on the component login gate (server RBAC is the real control).

Fix: Add a route guard for UX + defense-in-depth.

LOW Secrets in plaintext .env on disk

Evidence: Real Atlas creds + ADMIN_PASS in .env files. Confirmed NOT tracked by git (gitignored; ls-files/history clean) - good - but readable with host/disk access.

Fix: Use a managed secret store in prod; rotate the DB password given it appears in working files.

LOW Hardcoded F&B notification copy in Logistics mode

Evidence: Service worker push body hardcodes "Food is Ready!" regardless of BUSINESS_TYPE.

Fix: Parameterize by business type.

Controls verified IN PLACE (good)

- Helmet (HSTS preload, nosniff, frameguard, referrer-policy); x-powered-by off; trust proxy 1.
- bcrypt cost 12; rate limiting (login 5/15min skip-success, orders 60/min, global 300/min).
- Zod strips unknown keys (mass-assignment/BOPLA defense) + 422 field errors; escapeRegex guards ReDoS/regex-injection.
- CSRF: Origin allowlist on cookie auth routes; Bearer API is CSRF-immune. Refresh token hashed at rest, rotated, revocable.
- No XSS sinks in client (no dangerouslySetInnerHTML / innerHTML / eval); no hardcoded secrets in source; prod error handler hides stack/message.

11. Performance Audit

Frontend (vite build - executed, 4.9s)

Clean build with proper route-level code-splitting; heavy export libs load only on demand:

Chunk	Raw	Gzip	Loading
index (entry)	160 kB	53 kB	eager
AdminDashboard	225 kB	59 kB	lazy route
xlsx (SheetJS)	493 kB	161 kB	lazy (export only)
jspdf	400 kB	130 kB	lazy (PDF export)
html2canvas	200 kB	47 kB	lazy (PDF export)
LedgerTab	100 kB	19 kB	lazy

- **Good:** ~290 kB of export libs do not block first paint; entry + dashboard (~112 kB gz) is acceptable for a tablet LAN app.
- **Watch:** AdminDashboard at 59 kB gz is large for one route; further tab splitting would help cold load.

Backend

- **Good:** Mongo pool (maxPoolSize 20), fail-fast selection (10s), gzip, 40 schema indexes, atomic \$inc, bounded report ranges (<=92 days).
- **Watch:** analytics/dashboard partly client-side; some list endpoints build in-memory maps of all inventory (O(n)/request) - fine at SMB scale, not 10k+ SKUs.
- **Constraint:** order completion needs a replica set (Atlas OK; standalone Mongo fails). Not noted in docker-compose (which ships no Mongo service).

12. UI / UX Review

Partially verified live this audit: the app was logged into in a real browser and every admin tab was opened (see Section 15). Login, navigation, theming, the SUPER role badge, and empty states render correctly with zero console errors. Deeper UX (accessibility, responsive breakpoints) remains code-level only.

- **Verified live** - login screen + authenticated dashboard render; sidebar groups (Operations / Management), role-gated "Command Center (SUPER)", "No orders in any queue" empty state, Clock-In, auto-close timer; all 8 tabs mount without errors.
- **Theming** - data-theme (default/yellow/ocean) via CSS variables; black/olive brand confirmed in the live screenshot; tablet-first with large ($\geq 44\text{px}$) touch targets per the code.
- **Resilience** - top-level React ErrorBoundary with a reload affordance; Suspense loading fallback on lazy routes.
- **Not verified (recommend Lighthouse + axe)**: color contrast, keyboard accessibility, ARIA labeling, focus management, screen-reader behavior, responsive breakpoints across phone/tablet.

UI/UX status

Core flows are now browser-verified (login, navigation, all tabs, ledger views - 9/9 e2e). What remains unverified is accessibility and responsive correctness across device sizes, which need a Lighthouse + axe pass on a tablet viewport.

13. Database Review

- **Schema** - Mongoose models for Order, Product, Inventory, Category, User, ClientAccount, JournalEntry, Account, Counter, Settings, AuditLog, RefreshSession, and more; timestamps enabled; soft-delete via `isArchived`.
- **Indexes** - 40 index/unique declarations (orderId, businessType, idempotencyKey sparse, cashier, clientAccountId, etc.). Reasonable coverage for the main query paths.
- **Integrity** - balanced-journal assertion before commit; multi-document transactions on order completion (24 session-scoped ops); atomic counters prevent duplicate sequence numbers.
- **Tenancy** - businessType field stamped on docs + one-time backfill migration; not hard isolation.
- **Migrations** - hand-rolled in `runStartupTasks` (role backfill, businessType stamping, COA 4->6 digit code migration, counter sync), each guarded by a Settings flag. Works, but there is NO versioned/rollbackable migration framework.
- **Backup** - `scripts/backup-mongo.sh` + `restore-mongo.sh` provided; an automated/scheduled backup is not proven to run.

14. API Review

139 REST routes + 1 health endpoint, all enumerated. Consistent `{success, ...}` envelope, sensible status codes (401 vs 403 distinction, 422 validation, 404 fallback).

Concern	Assessment
REST consistency	GOOD - resource-oriented; consistent response shape
Status codes	GOOD - 200/401/403/404/422/500 used correctly; 401 vs 403 deliberate
AuthN/AuthZ	GOOD - every sensitive route guarded; public routes are menu reads + QR session + one IDOR exception
Validation	PARTIAL - Zod on key write routes; not universal

Rate limiting	GOOD - login/order/global limiters
Versioning	MISSING - no /v1 prefix
Documentation	MISSING - no OpenAPI/Swagger spec

Public (unauthenticated) routes - complete list

Route	Intent	Verdict
GET /health	Load-balancer health	OK
GET /api/products, /categories, /addons, /combos	Customer menu reads	OK except COGS/recipe leak (see Security)
POST /api/sessions/:id/heartbeat, /close	QR session keepalive	OK (public by nature)
POST /api/orders	Place order	OK - orderLimiter + verifyOrderAuth (QR session or staff token)
POST /api/users/login, /client-auth/login	Login	OK - loginLimiter + Zod
POST /api/auth/refresh, /logout	Token rotation	OK - requireTrustedOrigin + httpOnly cookie
GET /api/orders/:id	Order status poll	FINDING - no auth; exposes PII (IDOR)

Authorization tiers (verified across all 139)

- verifyToken + requireSuperAdmin - finance/journal/reports/accounts/users-mgmt/voids/inventory-destructive/admin tools.
- verifyToken + requireStaff - orders list, inventory ops, shifts, clock, categories, settings read.
- verifyToken + requireSuperOrAdmin - refunds. verifyClientToken - the 2 client-portal routes.

15. Testing Review

Suite	Result	Evidence
Server unit + integration (vitest)	PASS - 304/304, 20 files	Ran npm test (exit 0)
Server coverage (v8)	PARTIAL - 77.8% lines / 49.1% branch / 75% funcs	Ran npm run coverage (split below)
Pure lib coverage (ledger/tax/units/...)	GOOD - 99.1% lines / 89.1% branch	Co-located *.test.js per lib
Client build	PASS - clean	Ran vite build (exit 0)
Client unit tests	MISSING - none	No client unit runner configured
E2E (Playwright, real Chromium)	PASS - 9/9	EXECUTED this audit vs an ephemeral DB

Coverage split by area (verified - npm run coverage)

Area	Stmts	Branch	Funcs	Lines
server/lib (pure core)	97.2%	89.1%	97.2%	99.1%
authz.js / tax.js / reportRange.js	100%	100%	100%	100%
ledger.js	100%	94.7%	100%	100%

server.js (routes + orchestration)	71.3%	46.8%	73.1%	77.0%
ALL FILES	72.3%	49.1%	75.1%	77.8%

- The highest-risk logic (journal balancing, tax, authorization) is at ~100%. The gap is in server.js error/edge branches - and part of the "uncovered" server.js lines are the bootstrap/shutdown handlers (listen, SIGTERM, uncaughtException) deliberately skipped under test, so effective handler coverage is a bit higher than 46.8% branch implies.

Live browser verification (EXECUTED this audit)

The project's own Playwright suite was run in real headless Chromium against an EPHEMERAL in-memory MongoDB replica set (production Atlas never touched). **Result: 9/9 passing.** Verified end-to-end in a browser:

- Dual-token auth: login reaches the dashboard; access token NOT in localStorage; session survives reload (silent refresh); spamming reload keeps the user logged in; clearing the refresh cookie forces re-login.
- Navigation: ALL 8 admin tabs (Orders & POS, Inventory, Menu Setup, History & Shifts, Analytics, Accounting & Ledger, Pricing, Audit) mount with NO console/runtime errors; ledger sub-views (Journal, P&L, Balance Sheet, A/R, A/P) load.
- Smoke: login screen renders; dashboard renders post-login clean.

Live finding reproduced during this run

The first e2e attempt failed at login with "Network error" - the browser origin was not in the backend ALLOWED_ORIGINS, so CORS blocked every call and the app looked broken. Adding the origin fixed it and all 9 passed. This is a real, in-vivo demonstration of deployment finding #2: a wrong ALLOWED_ORIGINS makes the entire deployed app non-functional even though the code is correct.

Honest read: backend logic is well-tested (esp. money paths) with strong integration coverage incl. fault injection; the core UI is now BROWSER-VERIFIED (login, session, all tabs, ledger views, zero console errors). Remaining gaps: no client UNIT tests, deep frontend logic (cart math, exports, offline queue) is not automatically tested, and the e2e is not yet wired into CI. Blended automated-test confidence: **moderate-to-good (~66%)**.

16. Deployment Review

Capability	Status	Notes
Production build	GOOD	vite build clean; server node --check in CI
Containerization	GOOD	Dockerfiles (server + client/nginx) + docker-compose with healthcheck
CI/CD	PARTIAL	GitHub Actions builds+tests both apps; audit gate non-blocking; no CD
Config management	PARTIAL	.env.example thorough; live .env has weak secret + stray vars
Health checks	GOOD	/health reports DB state; compose healthcheck wired
Graceful shutdown	GOOD	SIGTERM/SIGINT drain; fatal handlers exit for supervisor restart
Logging	GOOD	Structured pino; request logging; /health noise filtered
Monitoring / crash reporting	PARTIAL	Sentry supported but inert unless SENTRY_DSN set
Backups / rollback	PARTIAL	Backup/restore scripts + GO_LIVE runbook; no scheduled backup or migration rollback
Secrets management	LOW	Plaintext .env; no managed secret store wired

DB provisioning

PARTIAL

compose ships NO mongo service - external Atlas (replica set) required

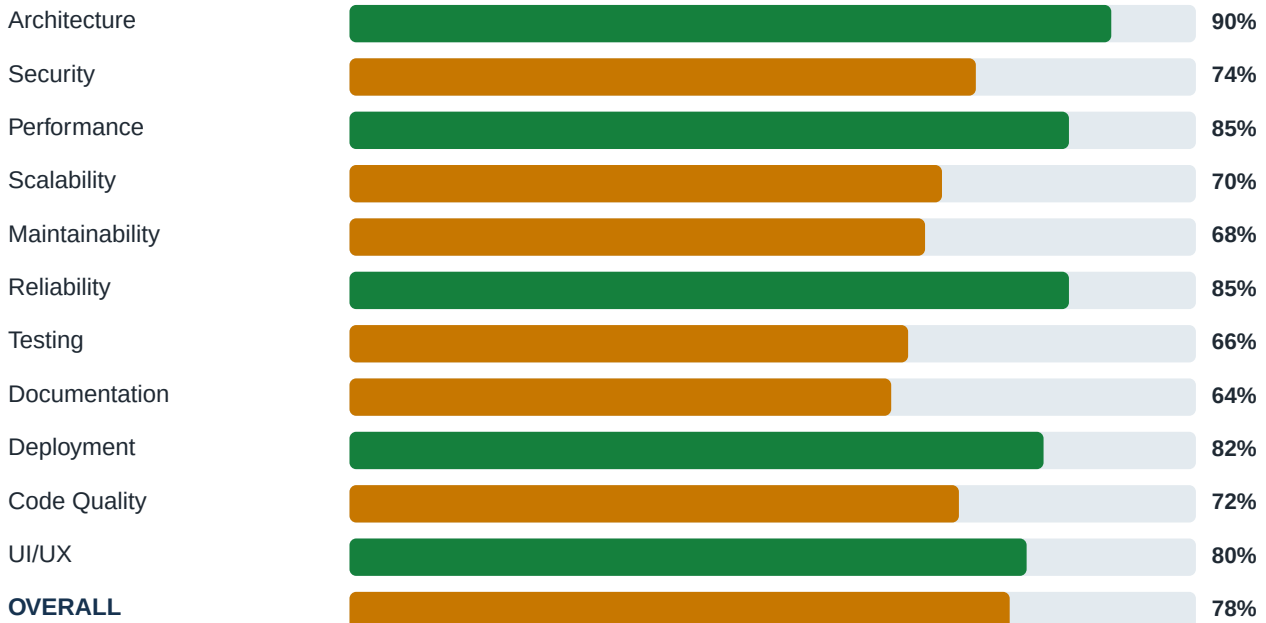
SSL / domain

PARTIAL

Terminated at Vercel/Railway/nginx; not configured in-repo

17. Production Readiness Ratings

Evidence-based estimates (0-100). Green ≥ 80 , amber 60-79, red < 60 .



Overall Production Readiness: 78%

Weighted toward a strong, well-tested backend and solid deployment scaffolding, pulled down by a weak live secret, an IDOR/PII exposure, unpatched high-severity dependencies, and maintainability/documentation debt. None of the blockers are architectural.

18. Deployment Decision

DECISION: YES - deployable to a single-tenant SMB, but ONLY after 3 must-fix blockers.

The system is functionally complete and operationally scaffolded for a real go-live. It is NOT safe to expose to the public internet in its current configuration until the blockers below are cleared - all small, well-understood fixes.

Must-fix blockers (before any live deployment)

CRITICAL 1. Replace the JWT secret

Action: Generate a strong UNIQUE per-deployment secret (openssl rand -hex 32), store in the host secret manager, add a boot strength check, rotate now (logs everyone out once).

HIGH 2. Patch dependencies

Action: npm audit fix in server/ and client/ to clear the high-severity ws DoS; make the CI audit gate blocking.

HIGH 3. Close the unauthenticated PII exposure

Action: Lock down GET /api/orders/:id and trim COGS/recipe from the public products response.

Strongly recommended before public/scale launch

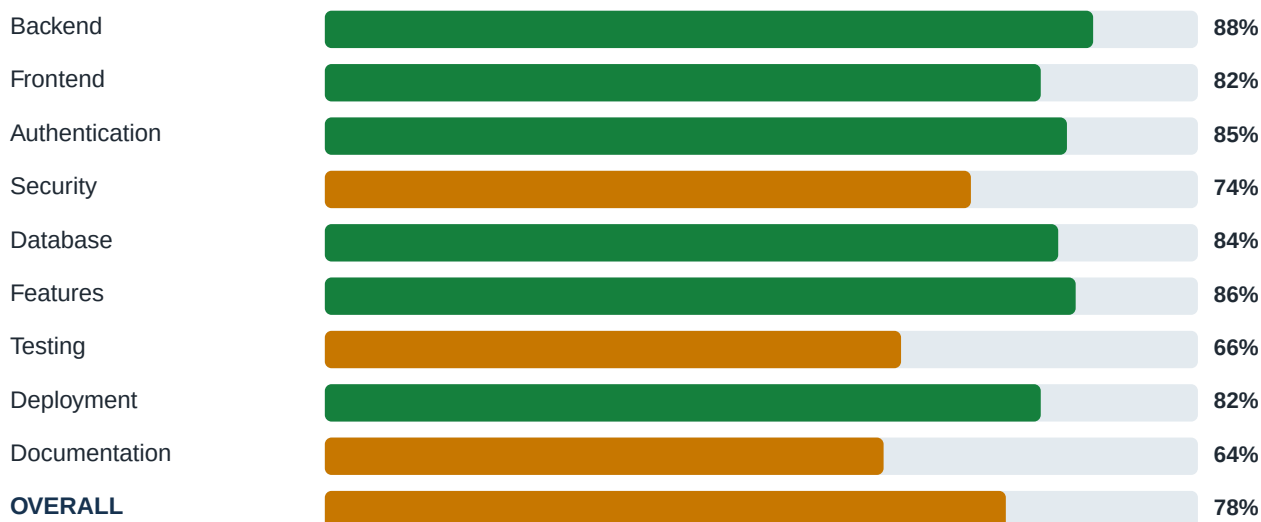
- Force-rotate the default/guessable admin password and the Atlas DB password.
- Add boot-time config validation (secret length, no default admin pass in prod, warn on empty origins).
- Commit the working-tree changes and let CI validate; reconcile the version number and tag a release.
- Wire SENTRY_DSN; verify an automated backup actually runs and restores.

19. Bug Prediction & Risk Analysis

Scenario	Likelihood	Predicted failure / risk
Public internet exposure	High	JWT forgery (weak secret) + order PII harvesting (IDOR) before other defenses matter
Socket.io DoS	Medium	ws memory-exhaustion advisory directly reachable until patched
Standalone/local MongoDB	High	Order completion throws - transactions need a replica set (Atlas only)
Years of data growth	Medium	Client-side analytics + in-memory product/inventory maps degrade; reports slow
High concurrency at counter	Low	Mitigated: atomic \$inc + atomic counters + idempotency keys prevent oversell/dupes
Offline -> reconnect sync	Medium	Queue replay relies on idempotency keys; needs real-world soak testing for edge dupes
Crash / uncaught exception	Low	Handled: drains + exits for supervisor restart; Sentry capture if configured
Unexpected user input	Low	Zod + escapeRegex + typed schemas cover most boundaries
Multi-branch expansion attempted	High	No real multi-tenancy; all clients receive all Socket events - data-bleed risk
Mongoose major upgrade	Medium	Deprecated { new: true } usage breaks on a future major

20. Completeness Analysis

Estimated completion by area:



Is it 100% complete? No - approximately 78% production-ready. The gap to "100%" is disciplined finishing work (Section 22 checklist), not research-level or architecturally risky.

21. Final Scorecard

Category	Score	Notes
Architecture	90%	Clean SPA/API/WS split; pure-logic core extracted & tested
Code Quality	72%	Strong patterns, but god files + no lint
Performance	85%	Code-split bundles, indexed DB, atomic ops; client-side analytics caveat
Security	74%	Strong design; weak live secret + IDOR + dep vulns drag it down
Scalability	70%	Single-tenant; client-side analytics; monolith
Maintainability	68%	6.7k-line server, 6.1k-line component, doc drift
Deployment	82%	Docker/CI/health/graceful shutdown; non-blocking audit, plaintext secrets
Documentation	64%	Abundant but drifted/contradictory; no API spec
Testing	66%	304 backend tests (77.8% lines) + 9/9 e2e executed; no client unit tests
UI/UX	80%	Core flows browser-verified (9/9 e2e + screenshot); a11y/responsive unverified
Reliability	85%	Transactions, graceful shutdown, fatal handlers, idempotency
OVERALL	78%	Release Candidate; deployable after 3 blockers

22. Final Verdict & 100% Completion Checklist

Classification: RELEASE CANDIDATE.

Beyond Beta - feature-complete for its single-tenant scope, well-tested on the backend, operationally packaged. Short of "Production Ready" only due to fixable config/exposure/dependency blockers; short of "Enterprise/Commercial multi-tenant Ready" due to single-tenancy, the monolithic frontend, light frontend testing, and documentation drift. Clear the 3 blockers and it becomes a legitimate single-tenant production deployment.

Critical (deploy blockers)

- [] Replace JWT_SECRET with a strong, unique-per-deployment value; add boot strength check.
- [] npm audit fix (server + client); make the CI audit gate blocking on high/critical.
- [] Authenticate / de-PII GET /api/orders/:id.

Major

- [] Strip COGS/recipe fields from the public /api/products response.
- [] Force-rotate default admin password + Atlas DB password; remove shipped defaults.
- [] Add boot-time config validation (secret length, prod origins, no default admin pass).
- [] Commit working-tree changes; reconcile version (1.0.0 / 0.5 / 0.2) and tag a release.
- [] Wire Sentry DSN in prod; prove backup + restore end-to-end.

Minor

- [] Add ESLint + Prettier + a CI lint step.
- [] Replace deprecated Mongoose { new: true } with { returnDocument: "after" }.
- [] Remove scripts_temp/ from the repo; de-duplicate showcase.html; gitignore generated PDFs.
- [] Reconcile/archive stale reports (AUDIT_OVERVIEW, SESSION_REPORT) that contradict current code.
- [] Parameterize push-notification copy by business type.

Recommended improvements

- [] Split server.js into route modules; continue AdminDashboard.jsx tab extraction.
- [] Add Playwright E2E (checkout, void, EOD) to CI; add client unit tests for cart/money math.
- [] Raise server branch coverage toward 70%+.
- [] Move heavy analytics aggregation server-side; paginate large lists.
- [] Publish an OpenAPI spec; add API versioning.

Optional enhancements

- [] True multi-tenancy + Socket.io room partitioning (unlocks multi-branch/franchise).
- [] Loyalty/CRM module; hardware (ESC/POS) certification matrix.
- [] Formal, versioned DB migration framework with rollback.

Closing statement

Semivra Libellus is a genuinely capable, above-average SMB POS/ERP with a security and accounting core most products in its class do not attempt. Its problems are concentrated and fixable: tighten three configuration/exposure items to deploy safely, then pay down monolith and documentation debt to make it maintainable and scalable. A strong Release Candidate - not yet a finished product.

Appendix A - What To Read Later

A curated reading list so you can verify this report and act on it without re-deriving context. Ordered by priority.

A.1 Read FIRST - to confirm the 3 blockers yourself

What to open	Why	Look for
server/.env (and the F&B .env)	Confirm the weak/reused JWT secret	JWT_SECRET=infu-pass-2026 (same in both)
server/server.js : 2085	The unauthenticated order-lookup (IDOR/PII)	app.get("/api/orders/:id", ...) with NO verifyToken
server/server.js : 1751-1797	COGS/recipe leak on the public menu	baseRecipe / costOverride still returned to anonymous
server/server.js : 44	Boot env check (presence only, not strength)	if (!MONGO_URI !JWT_SECRET) ...
npm audit (server/ and client/)	The 9 dependency advisories	high-severity ws DoS; run npm audit fix

A.2 Read to UNDERSTAND the system (the well-built parts)

- server/lib/*.js (+ the matching *.test.js) - the pure, fully-tested core: ledger, tax, units, expiry, chartOfAccounts, authz, reportRange. Start here; it is the clearest code in the repo.
- server/server.js auth middleware (verifyToken, verifyClientToken, requireStaff/SuperAdmin) and the order-completion transaction (around the mongoose.startSession() blocks) - the heart of correctness.
- client/src/lib/auth.js - the in-memory access token + silent refresh pattern (why tokens are NOT in

localStorage).

- `client/src/lib/pwa.js` + `client/public/sw.js` - the offline queue and service-worker caching/notifications.
- `server/test/*.integration.test.js` - the best living documentation of expected behavior (money math, fault injection, edge cases).

A.3 In-repo docs - trust map

Document	Use it for	Trust
DEPLOY.md, GO_LIVE.md, Makefile	Deploy steps, rollback runbook, ops commands	Trust (operational)
MANUAL.md, Semivra_Complete_Guide	End-user / operator how-to	Trust (usage)
PROJECT_CHARTER.md, README.md	Intent, scope, feature list	Mostly - version is stale (says v0.2)
AUDIT_OVERVIEW.md, SESSION_REPORT.md	Prior self-audit	CAUTION - stale: claims 75/81 tests, 0 vulns, token rewrite "not done", 4,661-line dashboard. All contradicted by current code.
The 8 committed *.pdf reports	Historical snapshots	CAUTION - point-in-time; may be outdated

Reconcile the docs

Several in-repo reports describe an earlier state of the project and now contradict the code (test counts, vuln counts, auth status, file sizes, version). Before sharing this project with anyone, archive or update those documents - conflicting "official" reports erode trust faster than a known gap.

A.4 External references for the findings

- OWASP Top 10 - A01 Broken Access Control (the IDOR) and A07 Identification & Authentication Failures (the JWT secret).
- OWASP Password Storage & Secrets Management cheat sheets; JWT best current practices (RFC 8725).
- Advisories to track: GHSA for `ws` (memory-exhaustion DoS), `dompurify` (Trusted-Types/ALLOWED_ATTR), `@opentelemetry/core` (baggage memory).
- MongoDB docs - multi-document transactions require a replica set (explains the standalone-Mongo risk).

A.5 Suggested order of work

- 1) Fix the 3 blockers (Section 18) -> 2) rotate admin + DB passwords -> 3) commit working tree + let CI run -> 4) add ESLint + blocking audit gate -> 5) reconcile version & docs -> 6) then start splitting the monoliths behind tests.